

Data Science Tools And Software

Dr. Laila M. Fatehy

Department Of Mathematics And Computer Science
Alexandria University

Review - Statistics

- Statistics refers to the mathematics and techniques with which we understand data and make sense of information. It helps us collect, organize, analyze, and interpret data to find patterns, trends, and relationships.
- Statistics serves as the backbone of data science providing tools and methodologies to extract meaningful insights from raw data.
- Data scientists rely on statistics – from cleaning messy datasets and creating powerful visualizations to building predictive models.

There are commonly two types of statistics:

- 1.Descriptive Statistics:** used to summarize and describe the features of a dataset by transforming the data into useful information.
- 2.Inferential Statistics:** it uses smaller data to conclude a larger group. It helps us predict and draw conclusions about a population.

Review - Descriptive Statistics

Descriptive statistics help us understand:

1. Where the data centers (Measures of Central Tendency ; Estimates of Location;)
2. How spread out the data is (Measure of Variability)

These measures are essential for understanding the distribution and variability of the data. When doing any kind of data analysis, it's important to have a clear understanding of these descriptive statistics.

Measures of Central Tendency

1. Mean: The average value.

Example: For the dataset [2, 4, 6, 8, 10], the mean is calculated as $((2+4+6+8+10)/5 = 6)$.

- **Weighted mean:** The sum of all values times a weight divided by the sum of the weights.
- **Trimmed mean:** The average of all values after dropping a fixed number of extreme values.

Example of Calculating a Trimmed Mean

- ✓ Suppose we have the following dataset of 10 values: 2, 3, 4, 5, 7, 8, 9, 10, 12, 15
- ✓ Let's find the 20% trimmed mean. Because there are ten values, we need to remove the smallest two values (2 and 3) and the largest two values (12 and 15), leaving us with the following six values: 4, 5, 7, 8, 9, 10
- ✓ Then we take the mean of these six values to find the trimmed mean, which is 7.1667.

2. **Median:** The value such that one-half of the data lies above and below.

Example: For the dataset [2, 4, 6, 8, 10], the median is 6.

3. **Mode:** The most frequently occurring value in a dataset.

Example: For the dataset [2, 4, 4, 6, 8, 10], the mode is 4.

Practical Applications of the Trimmed Mean

- **Financial Analysis:** When examining stock returns, extreme values (such as unusually high or low returns) can skew the mean, potentially leading to misinterpretation. By applying the trimmed mean, we obtain a more reliable **estimate** of the central tendency of returns.
- **Education Assessment:** In an examination where the scores of a few students are significantly higher or lower than the rest, the trimmed mean helps assess the overall performance by reducing the impact of outliers.

Example: A few rows of the state of population and murder rate by state

	State	Population	Murder rate	Abbreviation
1	Alabama	4,779,736	5.7	AL
2	Alaska	710,231	5.6	AK
3	Arizona	6,392,017	4.7	AZ
4	Arkansas	2,915,918	5.6	AR
5	California	37,253,956	4.4	CA
6	Colorado	5,029,196	2.8	CO
7	Connecticut	3,574,097	2.4	CT
8	Delaware	897,934	5.8	DE

Compute the mean, trimmed mean, and median for the population using R:

```
> state <- read.csv('state.csv')
> mean(state[['Population']])
[1] 6162876
> mean(state[['Population']], trim=0.1)      # trim=0.1 drops 10% from each end
[1] 4783697
> median(state[['Population']])
[1] 4436370
> weighted.mean(state[['Murder.Rate']], w=state[['Population']])
[1] 4.445834
```

Using python

To compute mean and median in *Python* we can use the *pandas* methods of the data frame. The trimmed mean requires the `trim_mean` function in `scipy.stats`:

```
state = pd.read_csv('state.csv')
state['Population'].mean()
trim_mean(state['Population'], 0.1)
state['Population'].median()
np.average(state['Murder.Rate'], weights=state['Population']) # to calculate the weighted mean
```

- The basic metric for location is the mean, but it can be sensitive to extreme values (outlier).
- Other metrics (median, trimmed mean) are less sensitive to outliers and unusual distributions and hence are more robust.

Review - Descriptive Statistics

Measures of Dispersion

1. Range: The difference between the maximum and minimum values.

Example: For the dataset [2, 4, 6, 8, 10], the range is (10-2=8).

2. Variance: A measure of how spread out the numbers in a data set are.

$$S^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

3. Standard Deviation: The square root of the variance, providing a measure in the same units as the original data.

$$S.D. = S = \sqrt{\text{variance}}$$

- Variance and standard deviation are the most widespread and routinely reported statistics of variability.
- Both are sensitive to outliers.
- The standard deviation is much easier to interpret than the variance since it is on the same scale as the original data.
- ***Other estimates of variability:***

$$\text{Mean absolute deviation} = \frac{\sum_{i=1}^n |x_i - \bar{x}|}{n}$$

$$\text{Median absolute deviation} = \text{Median}(|x_1 - m|, |x_2 - m|, \dots, |x_N - m|)$$

- A robust estimate of variability is the median absolute deviation from the median or MAD where m is the median. MAD is not influenced by extreme values (like median)
- Working with squared values is much more suitable than absolute values, especially for statistical models

Example: *Variability Estimates of State Population*

	State	Population	Murder rate	Abbreviation
1	Alabama	4,779,736	5.7	AL
2	Alaska	710,231	5.6	AK
3	Arizona	6,392,017	4.7	AZ
4	Arkansas	2,915,918	5.6	AR
5	California	37,253,956	4.4	CA
6	Colorado	5,029,196	2.8	CO
7	Connecticut	3,574,097	2.4	CT
8	Delaware	897,934	5.8	DE

Using R's built-in functions for the standard deviation, the interquartile range (IQR), and the median absolute deviation from the median (MAD), we can compute estimates of variability for the state population data:

```
> sd(state[['Population']])  
[1] 6848235  
> IQR(state[['Population']])  
[1] 4847308  
> mad(state[['Population']])  
[1] 3849870
```

Using python

The `pandas` data frame provides methods for calculating standard deviation and quantiles. Using the quantiles, we can easily determine the IQR. For the robust MAD, we use the function `robust.scale.mad` from the `statsmodels` package:

```
state['Population'].std()  
state['Population'].quantile(0.75) - state['Population'].quantile(0.25)  
robust.scale.mad(state['Population'])
```

The following code shows how to calculate these descriptive statistics using Python:

```
import numpy as np
# Generate 10 random numbers between 0 and 100
dataset = np.random.rand(10) * 100

# Mean
mean_value = np.mean(dataset)

# Median
median_value = np.median(dataset)

# Mode
from scipy import stats          # import scipy.stats as stats
mode_value = stats.mode(dataset)[0]

# Range
range_value = np.max(dataset) - np.min(dataset)

# Variance
variance_value = np.var(dataset)

# Sample Variance, note the `ddof` parameter set to 1 for sample variance
sample_variance_value = np.var(dataset, ddof=1)

# Standard Deviation
std_dev_value = np.std(dataset)

print(f"Mean: {mean_value}")
print(f"Median: {median_value}")
print(f"Mode: {mode_value}")
print(f"Range: {range_value}")
print(f"Variance: {variance_value}")
print(f"Sample Variance: {sample_variance_value}")
print(f"Standard Deviation: {std_dev_value}")
```

Review - Statistics

- **Random Variables**

A random variable is a variable whose possible values are outcomes of a random phenomenon.

- **Types of Random Variables:**

Discrete Random Variable: A random variable that can take on a countable number of distinct values.

Continuous Random Variable: A random variable that can take on an infinite continuum of possible values.

- **Probability Mass Function (PMF)**

A function that gives the probability that a discrete random variable is exactly equal to some value.

- **Probability Density Function (PDF)**

A function that describes the relative likelihood for a continuous random variable to take on a given value.

- **Cumulative Distribution Function (CDF)**

A function that gives the probability that a random variable is less than or equal to a certain value . $F(x)$

Probability Distributions

- **Discrete Probability Distribution:** Describes the probability of each possible outcome of a discrete random variable.
- **Continuous Probability Distribution:** Describes the probability of each possible outcome of a continuous random variable.

Common distributions:

- **Binomial Distribution:** Describes the number of successes in a fixed number of independent Bernoulli trials, each with the same probability of success.

$$b(x; n, p) = f(x) = \binom{n}{x} p^x q^{n-x} \quad \text{for } x = 0, 1, 2, \dots, n.$$

- **Poisson Distribution:** Describes the number of events occurring in a fixed interval of time or space, given a constant mean rate of occurrence.

$$f(x; \lambda) = \frac{e^{-\lambda} \lambda^x}{x!} \quad \text{for } x = 0, 1, 2, \dots$$

- **Exponential Distribution:** Describes the time between events in a Poisson point process, i.e., a process in which events occur continuously and independently at a constant average rate.

$$f(x; \theta) = \begin{cases} \frac{1}{\theta} e^{-x/\theta} & \text{for } x > 0 \\ 0 & \text{otherwise} \end{cases}$$

- ***Normal Distribution***

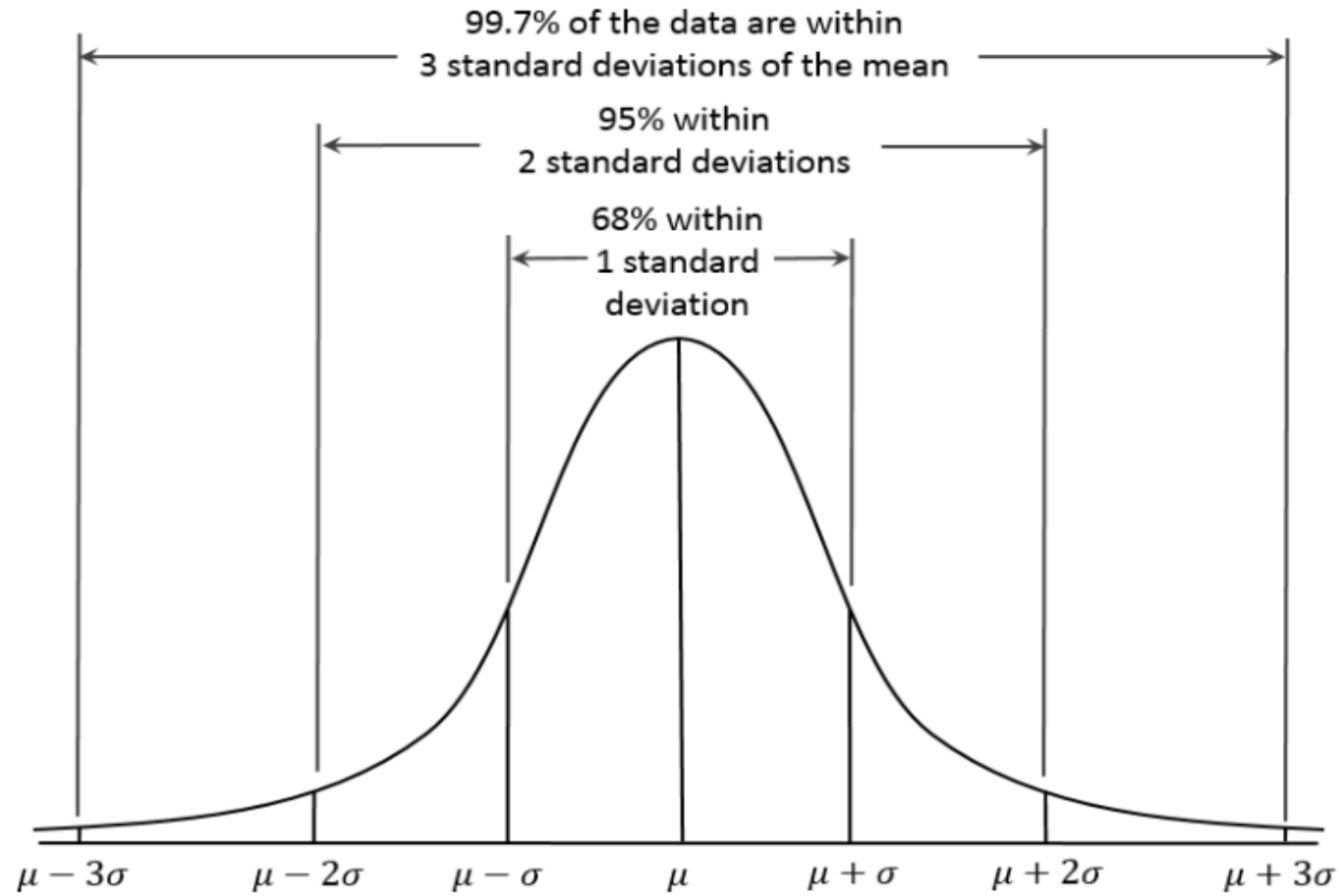
- Normal distribution, also known as the ***Gaussian distribution***, is a probability distribution that is symmetric about the mean.
- The fact that distributions of ***sample statistics*** are often normally shaped has made it a powerful tool in the development of mathematical formulas that approximate those distributions.
- **For a data scientist**, a dataset that satisfies a normal distribution is suitable for building models, that is the reason of most machine learning models work well with normally distributed data.

Key terms for normal distribution:

Standardize: Subtract the mean and divide by the standard deviation.

z-score: The result of standardizing an individual data point.

Standard normal: A normal distribution with mean = 0 and standard deviation = 1.



In a normal distribution, 68% of the data lies within one standard deviation of the mean, and 95% lies within two standard deviations.

Test of Normality

- Normality is an important assumption for many Machine Learning algorithms, especially those that rely on statistical inference, such as linear regression, ANOVA, or t-tests.
- Normality means that the data follows a normal distribution, also known as a bell curve, where most of the values are clustered around the mean and the tails are symmetrical.
- Testing for normality can help you choose the right algorithm, transform your data, or validate your results.
- **Three common methods:** graphical, numerical, and statistical are used to test normality.

1. Graphical Methods

- Graphical methods are visual ways. They can give you a quick sense of how your data is distributed and whether it deviates from the normal shape.
- Some of the most popular graphical methods are **histograms**, **box plots**, and **Q-Q plots**

Test of Normality- Graphical Methods

- **A histogram:** is a bar chart that shows the frequency of each value in your data. A normal histogram should have a single peak in the middle and taper off symmetrically on both sides. If the histogram is roughly “bell-shaped”, then the data is assumed to be normally distributed.
- **A box plot:** is a diagram that shows the median, quartiles, and outliers of your data. A normal box plot should have a symmetrical box with no extreme outliers.
- **A Q-Q plot:** is a scatter plot that compares your data to a theoretical normal distribution. A normal Q-Q plot should have the points aligned along a 45-degree line.

Histogram

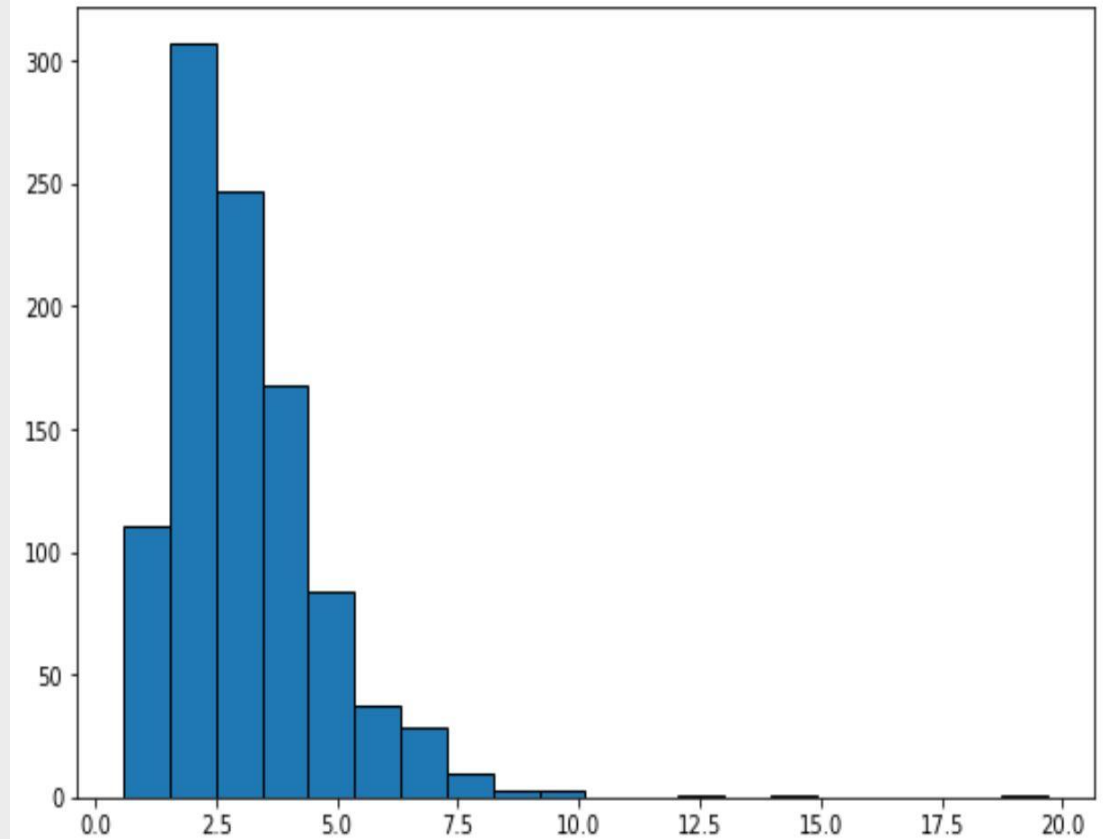
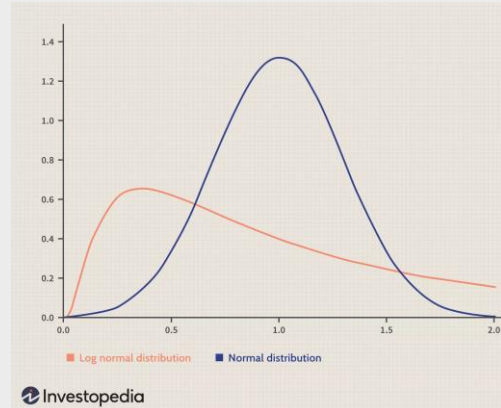
The following code shows how to create a histogram for a dataset that follows a **log-normal distribution**

```
import math
import numpy as np
from scipy.stats import lognorm
import matplotlib.pyplot as plt

#make this example reproducible
np.random.seed(1)

#generate dataset that contains 1000 log-normal distributed values
lognorm_dataset = lognorm.rvs(s=.5, scale=math.exp(1), size=1000)

#create histogram to visualize values in dataset
plt.hist(lognorm_dataset, edgecolor='black', bins=20)
```



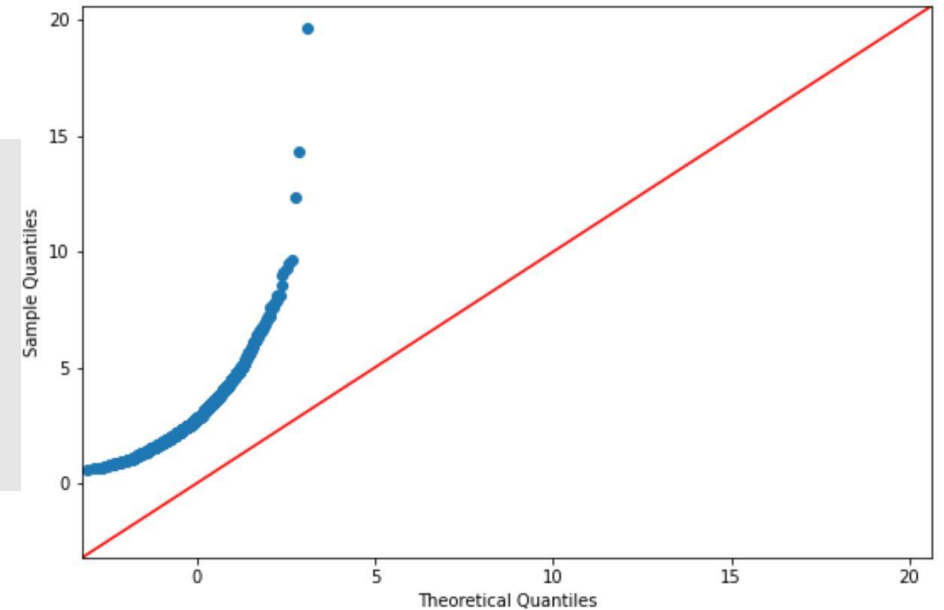
- By simply looking at this histogram, we can tell the dataset does not display a “bell-shape” and is **not** normally distributed.

QQ-plot

To create a Q-Q plot for the previous generated data from **log-normal** distribution:

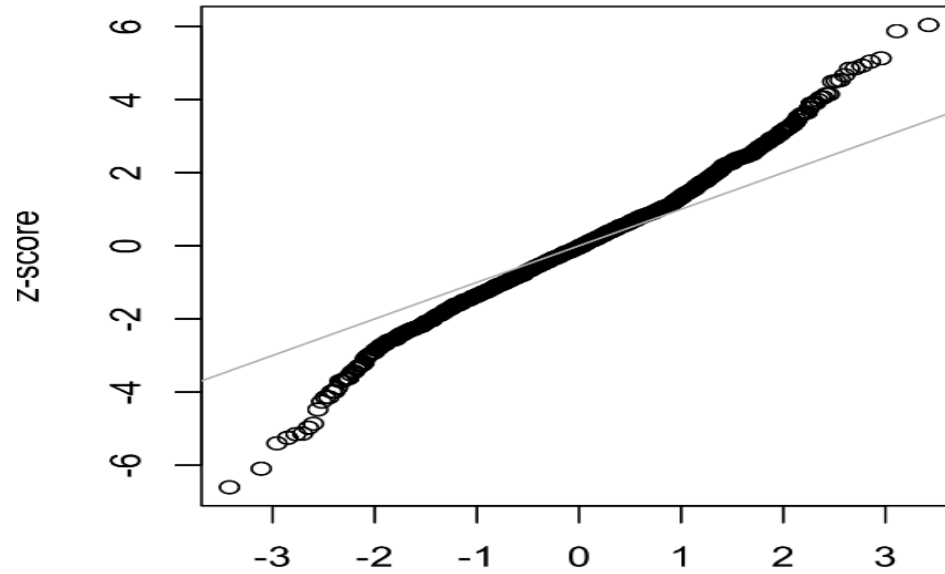
```
#To create a Q-Q plot for the previous generated data from  
# log-normal distribution:
```

```
import statsmodels.api as sm  
fig= sm.qqplot(lognorm_dataset,line= '45')  
plt.show()
```



- If the points on the plot fall along a straight diagonal line, then we typically assume a dataset is normally distributed.
- However, the points on this plot clearly don't fall along the red line, so we would **not** assume that this dataset is normally distributed.

Q: Interpretate the given QQ-plot of the returns of Netflix

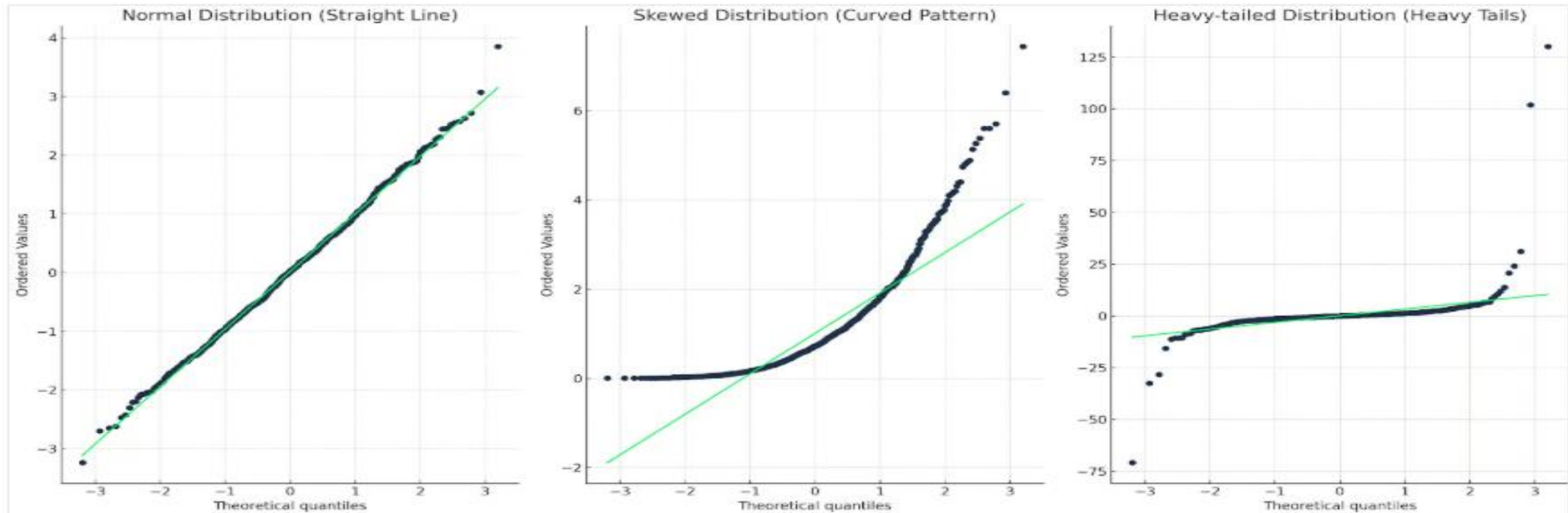


Answer: the points are close to the line for the data within one standard deviation of the mean. Tukey refers to this phenomenon as data being “normal in the middle” but having much longer tails.

In QQ-Plot, there are about three things that we look for:

- **Straight Line:** Data aligns well with the theoretical distribution.
- **Curved Patterns:** Indicate skewed data or non-normal distributions.
- **Heavy or Light Tails:** If points deviate at the ends, the data might have heavier or lighter tails than expected.

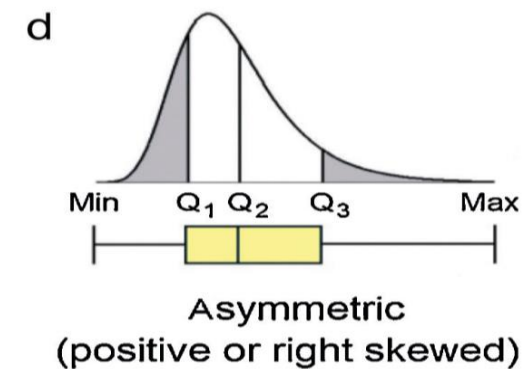
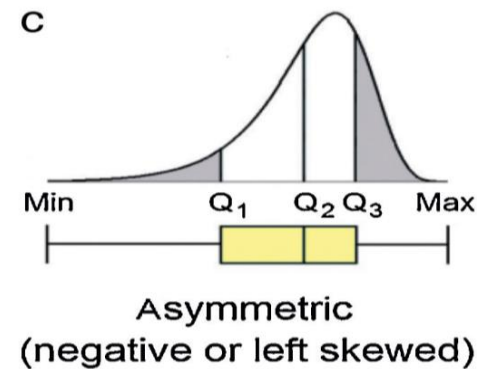
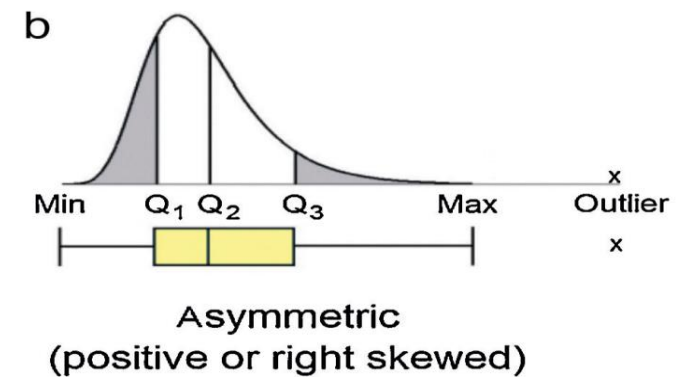
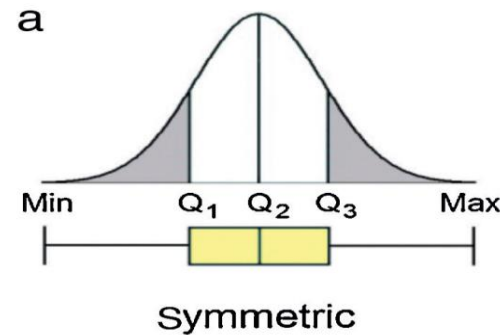
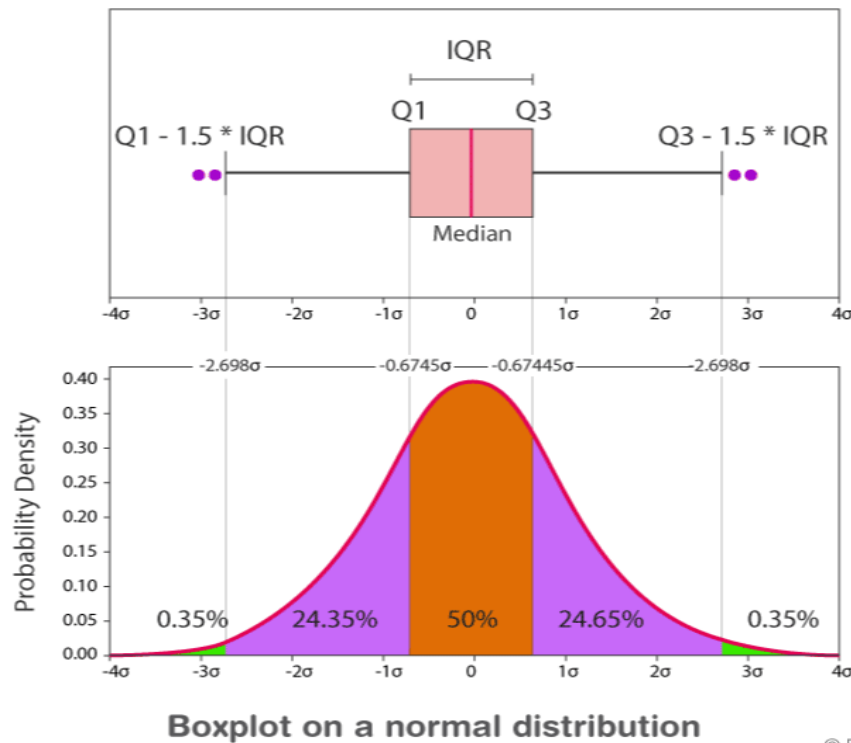
Let's show with an example for each:

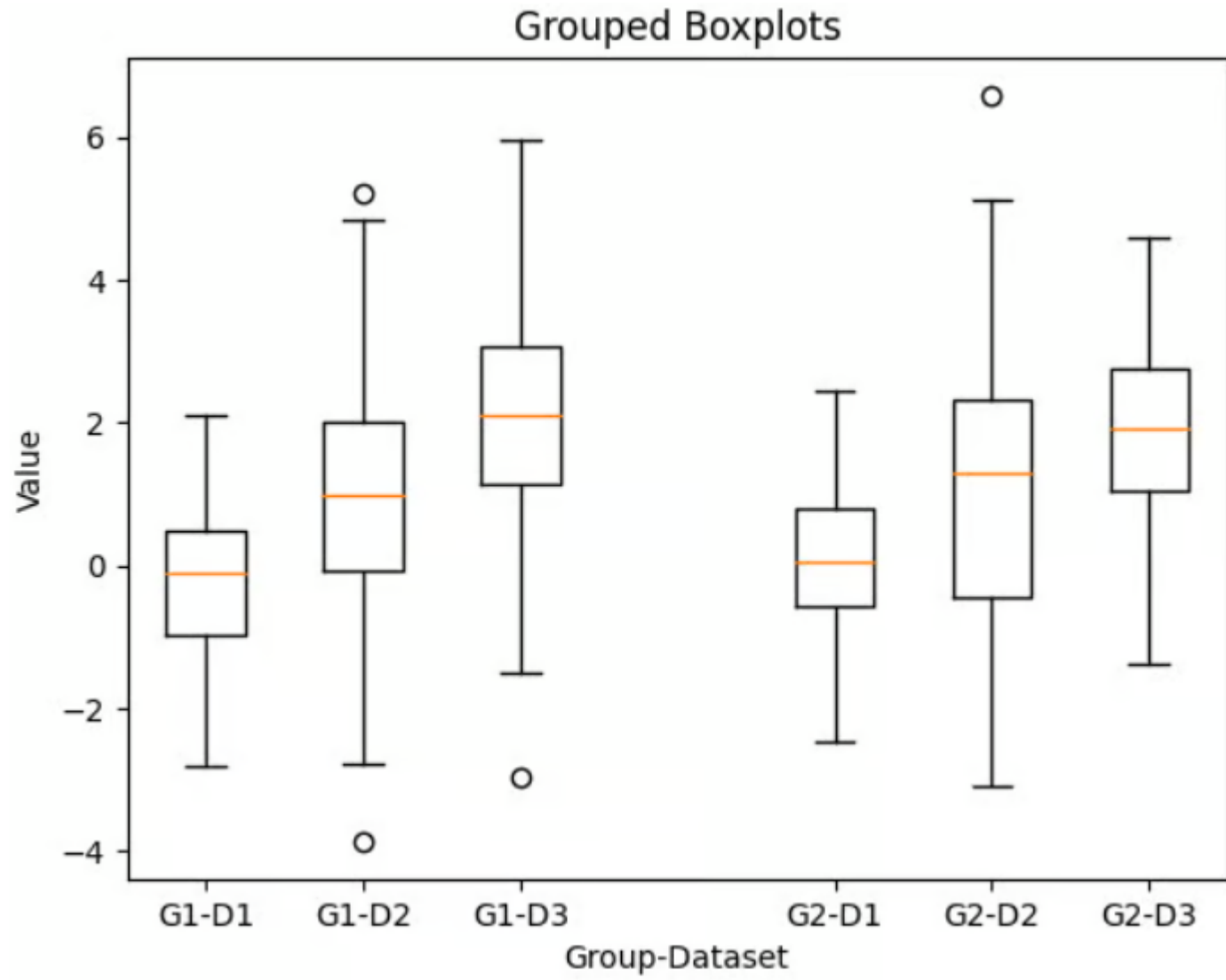


BoxPlot

The graph below shows a standard normal probability density function ruled into four quartiles, and the box plot you would expect if you took a very large sample from that distribution.

`plt.boxplot(data)`



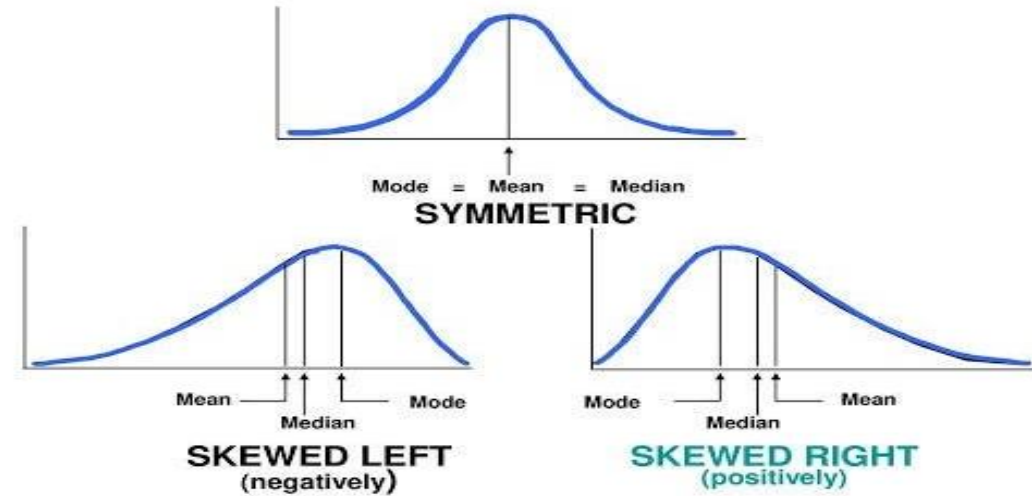
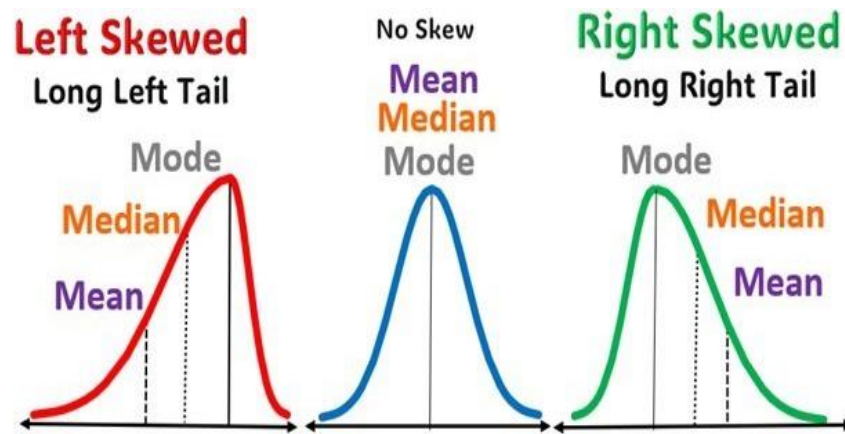


Test of Normality

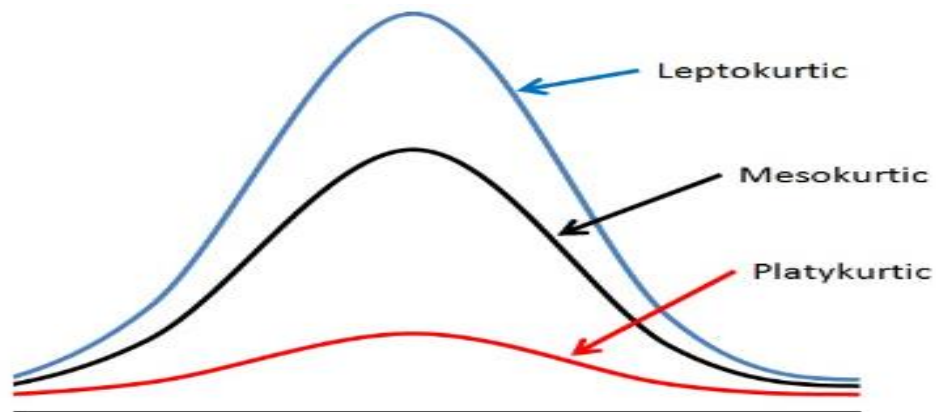
2. Numerical Methods

- Numerical methods are quantitative measures of how normal your data is.
- They can give you a more precise and objective assessment of your data's normality than graphical methods.
- Some of the most common numerical methods are **skewness**, **kurtosis**, and **mean-median difference**.
- **Skewness** is a measure of how asymmetric your data is. A normal distribution has a skewness of **zero**, meaning that it is perfectly balanced. A positive skewness means that your data has a longer right tail, while a negative skewness means that it has a longer left tail.
- **Kurtosis** is a measure of how peaked or flat your data is. A normal distribution has a kurtosis of **three**, meaning that it has a moderate peak and moderate tails. A higher kurtosis means that your data has a sharper peak and fatter tails, while a lower kurtosis means that it has a flatter peak and thinner tails.
- **Mean-median difference** is a measure of how far apart the mean and the median of your data are. A normal distribution has a mean-median difference of **zero**, meaning that they are equal. A large mean-median difference means that your data is skewed.

Skewness



Kurtosis



> 3

value of kurtosis = 3

< 3

Test of Normality

3. Statistical Methods

- Statistical methods are hypothesis tests that evaluate whether your data comes from a normal distribution or not.
- They can give you a more formal verification of your data's normality than graphical or numerical methods.
- Some of the most widely used statistical methods are **Shapiro-Wilk test**, **Kolmogorov-Smirnov test**, and **Anderson-Darling test**.
- These tests compare your data to a normal distribution and calculate a [p-value](#), which is the probability of observing your data or more extreme data if the null hypothesis of normality is true.

Test of Normality- Statistical Methods

- **Perform a Shapiro-Wilk Test:** if $p\text{-value} > \alpha = .05$, then the data is assumed to be normally distributed.

```
import math
import numpy as np
from scipy.stats import shapiro
from scipy.stats import lognorm

#make this example reproducible
np.random.seed(1)

# generate dataset that contains 1000 log-normal distributed values
lognorm_dataset = lognorm.rvs(s=.5, scale=math.exp(1), size=1000)

# perform Shapiro-Wilk test for normality
shapiro(lognorm_dataset)
```

```
ShapiroResult(statistic=0.8573321719184391, pvalue=3.880399896267678e-29)
```

Since the p-value is less than .05, we reject the null hypothesis of the Shapiro-Wilk test. This means we have sufficient evidence to say that the sample data does not come from a normal distribution.

Test of Normality- Statistical Methods

- **Perform a Kolmogorov-Smirnov Test:** if $p\text{-value} > \alpha = .05$, then the data is assumed to be normally distributed.

```
import math
import numpy as np
from scipy.stats import kstest
from scipy.stats import lognorm
#make this example reproducible
np.random.seed(1)

# generate dataset that contains 1000 log-normal distributed values
lognorm_dataset = lognorm.rvs(s=.5, scale=math.exp(1), size=1000)

# perform Kolmogorov-Smirnov test for normality
kstest(lognorm_dataset, 'norm')

KstestResult(statistic=0.84125708308077, pvalue=0.0,
             statistic_location=1.2773298091955965, statistic_sign=-1)
```

Since the p-value is less than .05, we reject the null hypothesis of the Kolmogorov-Smirnov test. This means we have sufficient evidence to say that the sample data does not come from a normal distribution.

Hypothesis Testing in Data Science

- In data science, hypothesis testing is **used to validate assumptions or claims about data**. It helps data scientists determine whether observed patterns are statistically significant or not.
- Hypothesis method *compares two opposite statements about a population and uses sample data to decide which one is more likely to be correct*.
- To test this assumption, we first take a sample from the population and analyze it and use the results of the analysis to decide if the claim is valid or not.

Steps in Hypothesis Testing

1. **State the Hypotheses:** Define H_0 and H_a .
2. **Choose the Significance Level:** Commonly used levels are 0.05 or 0.01.
3. **Calculate the Test Statistic:** Depending on the distribution and type of test, this could be a Z-score, t-score, etc.
4. **Determine the p-value:** The probability of observing the data if H_0 is true.
5. **Make a Decision:** If p-value < alpha, reject H_0 ; otherwise, do not reject H_0 .

```

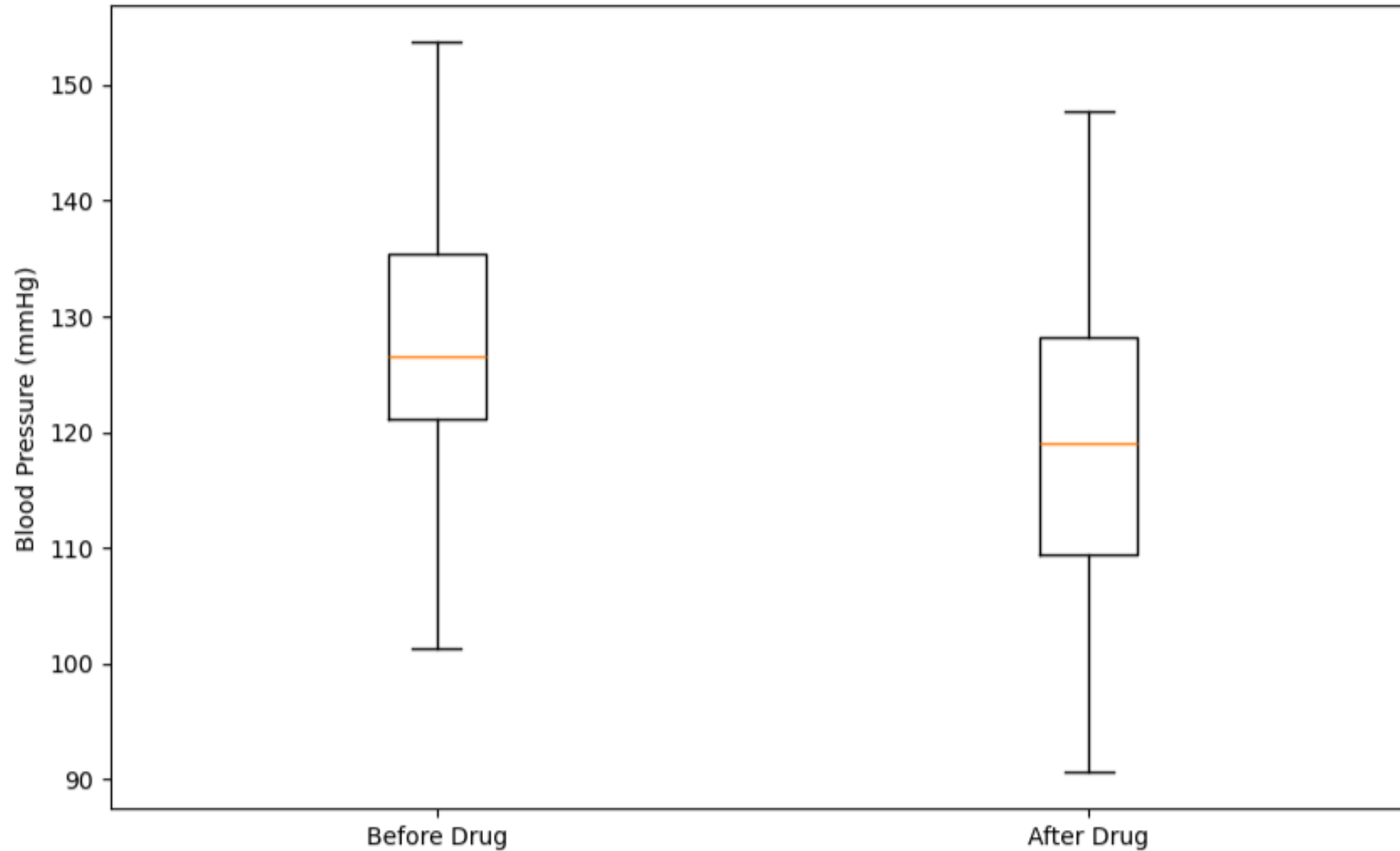
# Hypotheses Testing Example: (004-statistics.jpynb)
import numpy as np
from scipy import stats
import matplotlib.pyplot as plt
# Set random seed for reproducibility
np.random.seed(42)
# Sample data before and after taking the drug
before_drug = np.random.normal(loc=130, scale=15, size=30) # Mean 130, SD 15
after_drug = np.random.normal(loc=120, scale=15, size=30) # Mean 120, SD 15
# Print sample data
print("Before Drug:", before_drug) # print(f"Before Drug: {before_drug}" )
print("After Drug:", after_drug)
# Plot the data for visualization
plt.figure(figsize=(10, 6))
plt.boxplot([before_drug, after_drug], tick_labels=['Before Drug', 'After Drug'])
plt.title('Blood Pressure Before and After Taking the Drug')
plt.ylabel('Blood Pressure (mmHg)')
plt.show()

```

```
# Perform paired t-test
t_statistic, p_value = stats.ttest_rel(before_drug, after_drug)
print(f"T-statistic: {t_statistic}")
print(f"P-value: {p_value}")
# Set significance level
alpha = 0.05

# Make decision
if p_value < alpha:
    print("Reject the null hypothesis: The drug has an effect on blood pressure.")
else:
    print("Do not reject the null hypothesis: There is no significant effect of the drug on blood pressure.")
```


Blood Pressure Before and After Taking the Drug



T-statistic: 2.671799786487767

P-value: 0.012246799872481404

Reject the null hypothesis: The drug has an effect on blood pressure.

Real life Examples of Hypothesis Testing

Imagine a pharmaceutical company has developed a new drug that they believe can effectively lower blood pressure in patients with hypertension. Before bringing the drug to market they need to conduct a study to see its impact on blood pressure.

Data:

Before Treatment: 120, 122, 118, 130, 125, 128, 115, 121, 123, 119

After Treatment: 115, 120, 112, 128, 122, 125, 110, 117, 119, 114

Perform the required test to see the efficiency of the drug.

Central Limit Theorem (CLT)

The Central Limit Theorem (CLT) is one of the most fundamental and powerful concepts in statistics and plays a critical role in data science. It provides a foundation for analyzing data, making inferences about populations (such as hypothesis testing and confidence interval estimation), and building predictive models.

The Central Limit Theorem states that *"if we repeatedly take random samples from any population the distribution of their means will tend to be normal (bell-shaped) as the sample size increases and it is independent of the population's original distribution."*

Theorem (CLT): Let $X_1, X_2, \dots, X_n$ be i.i.d. random variables with mean μ and variance σ^2 . Then, as $n \rightarrow \infty$, the distribution of the standardized sum Z_n converges to a standard normal distribution. That is, $Z_n \rightarrow N(0,1)$

CLT in Finance: Portfolio Returns

In finance, the CLT is often used to model the returns of a portfolio composed of many assets. Suppose a portfolio consists of n assets, each with returns $R_1, R_2, \dots, R_n$, which are i.i.d. random variables with mean μ and variance σ^2 . The total return of the portfolio is:

$$R_{portfolio} = \frac{R_1 + R_2 + \dots + R_n}{n}$$

By the CLT, for large n , the distribution of $R_{portfolio}$ is approximately normal: $R_{portfolio} \sim N(\mu, \frac{\sigma^2}{n})$

Example: Finance: Portfolio Returns

We simulate the returns of a portfolio composed of many assets and demonstrate that the distribution of the portfolio's average return converges to a normal distribution:

- We simulate returns for n_assets , each with a mean return of **mu** and standard deviation **sigma**.
- The portfolio's average return is computed across all assets.
- The histogram of portfolio returns approximates a normal distribution, as predicted by the CLT.

```

import numpy as np
import matplotlib.pyplot as plt
import scipy.stats as stats

# Parameters
n_assets = 1000 # Number of assets in the portfolio
n_samples = 10000 # Number of simulations
mu, sigma = 0.05, 0.20 # Mean and standard deviation of individual asset returns

# Simulate portfolio returns
np.random.seed(42)
returns = np.random.normal(mu, sigma, (n_samples, n_assets))
portfolio_returns = np.mean(returns, axis=1) # Average return across assets

# Plot histogram of portfolio returns
plt.figure(figsize=(10, 6))
plt.hist(portfolio_returns, bins=50, density=True, alpha=0.6, color='b',
label='Portfolio Returns')

# Overlay the theoretical normal distribution
x = np.linspace(mu - 0.1, mu + 0.1, 1000)
plt.plot(x, stats.norm.pdf(x, mu, sigma / np.sqrt(n_assets)), 'r-', lw=2,
label='Normal Distribution')

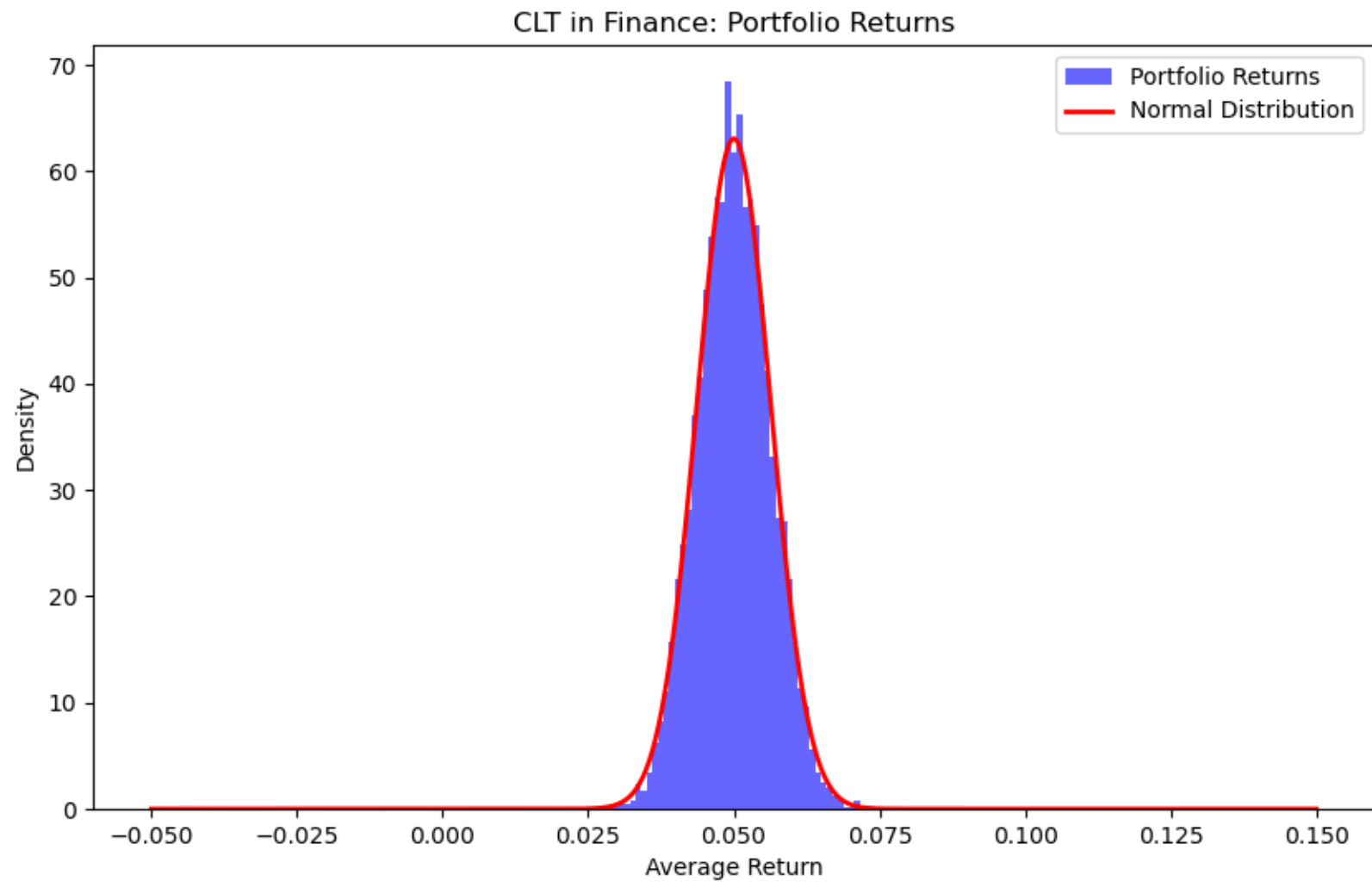
plt.title('CLT in Finance: Portfolio Returns')
plt.xlabel('Average Return')
plt.ylabel('Density')
plt.legend()
plt.show()

```

Notes:

❑ **Financial Portfolio** is defined as a collection of assets held by an investor, and it is a key concept in portfolio management.

❑ **Value-at-Risk (VaR)** is a risk measure that quantifies the maximum loss that a portfolio is likely to experience over a given time period with a certain level of confidence.



CLT in Media: Audience Engagement Metrics

In media, the CLT is used to analyze audience engagement metrics, such as the average time spent on a website or app. Suppose $x_1, x_2, \dots, x_n$ represent the time spent by n users on a platform, where each x_i is i.i.d. with mean μ and variance σ^2 . The average time spent by users is:

$$\bar{X} = \frac{x_1 + x_2 + \dots + x_n}{n}$$

By the CLT, for large n , the distribution of $\bar{X}$ is approximately normal:

$$\bar{X} \sim N\left(\mu, \frac{\sigma^2}{n}\right)$$

This allows media companies to construct confidence intervals for average engagement metrics and make data-driven decisions about content strategy.

Example: Media: Audience Engagement Metrics

We simulate the average time spent by users on a platform and show that the distribution of the average time converges to a normal distribution.

- We simulate engagement times for `n\_users` using an exponential distribution (non-normal).
- The average engagement time across users is computed.
- The histogram of average engagement times approximates a normal distribution, as predicted by the CLT.


```

import numpy as np
import matplotlib.pyplot as plt
import scipy.stats as stats

# Parameters
n_users = 1000 # Number of users
n_samples = 10000 # Number of simulations
mu, sigma = 5.0, 5.0 # Mean and standard deviation of time spent (in minutes)

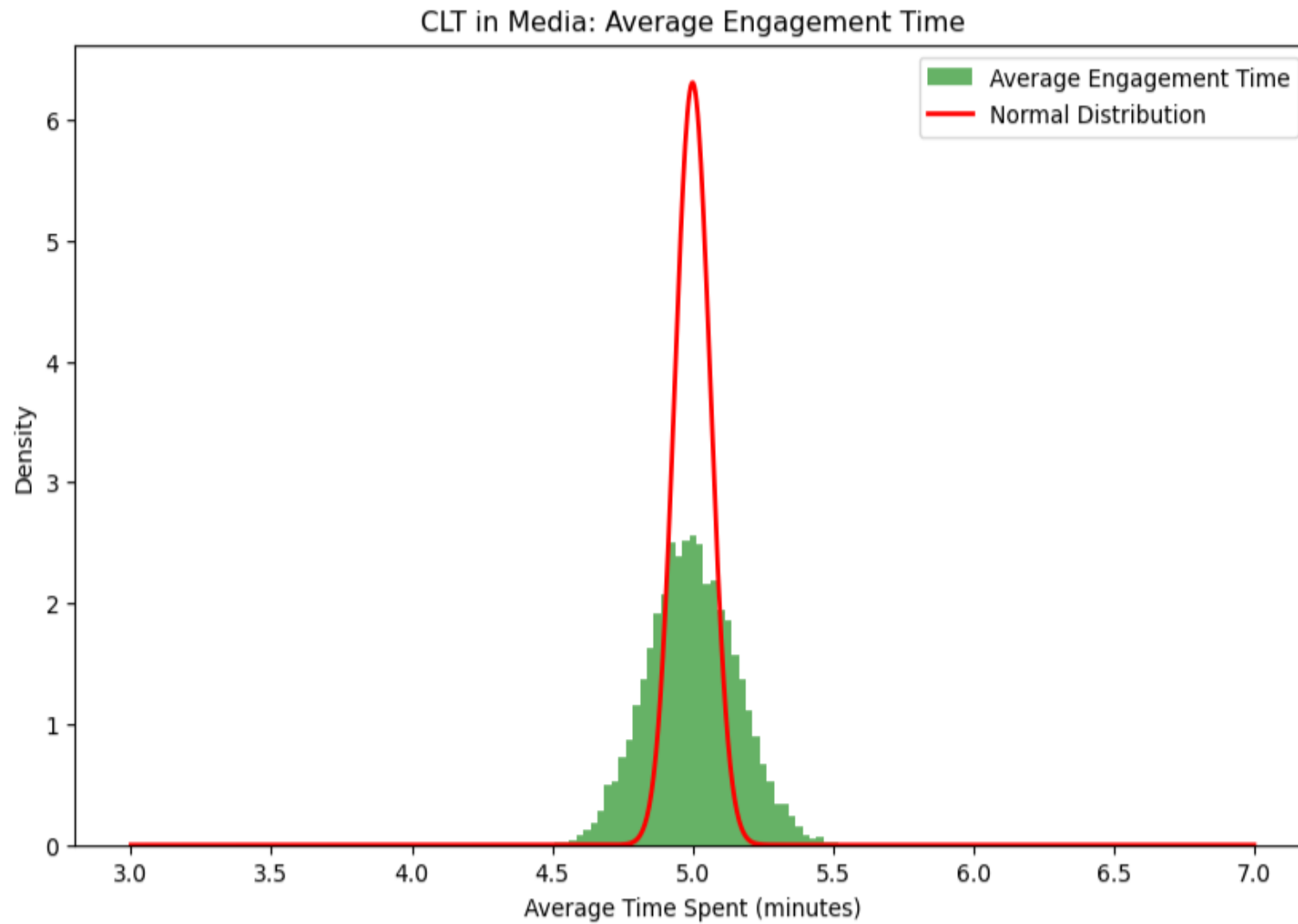
# Simulate user engagement times
np.random.seed(42)
engagement_times = np.random.exponential(mu, (n_samples, n_users)) # Exponential distribution
average_times = np.mean(engagement_times, axis=1) # Average time across users

# Plot histogram of average engagement times
plt.figure(figsize=(10, 6))
plt.hist(average_times, bins=50, density=True, alpha=0.6, color='g', label='Average Engagement Time')

# Overlay the theoretical normal distribution
x = np.linspace(mu - 2, mu + 2, 1000)
plt.plot(x, stats.norm.pdf(x, mu, sigma / np.sqrt(n_users)), 'r--', lw=2, label='Normal Distribution')

plt.title('CLT in Media: Average Engagement Time')
plt.xlabel('Average Time Spent (minutes)')
plt.ylabel('Density')
plt.legend()
plt.show()

```



- **CLT in Healthcare: Drug Efficacy Trials**

In healthcare, the CLT is used to analyze the results of clinical trials. Suppose a drug trial involves n patients, and $x_1, x_2, \dots, x_n$ represent the reduction in a specific health metric (e.g., blood pressure) for each patient. Assume that each x_i is i.i.d. with mean μ (the true average effect of the drug) and variance σ^2 . The sample mean reduction is:

$$\bar{X} = \frac{x_1 + x_2 + \dots + x_n}{n}$$

By the CLT, for large n , the distribution of $\bar{X}$ is approximately normal:

$$\bar{X} \sim N\left(\mu, \frac{\sigma^2}{n}\right)$$

This allows researchers to perform hypothesis tests (e.g., testing whether $\mu > \mu_0$) and construct confidence intervals for the drug's efficacy.

Example: Healthcare: Drug Efficacy Trials

We simulate the average reduction in blood pressure for patients in a clinical trial and demonstrate that the distribution of the average reduction converges to a normal distribution.

- We simulate blood pressure reductions for `n\_patients` using a gamma distribution (non- normal).
- The average reduction across patients is computed.
- The histogram of average reductions approximates a normal distribution, as predicted by the CLT.

```

import numpy as np
import matplotlib.pyplot as plt
import scipy.stats as stats

# Parameters
n_patients = 500 # Number of patients
n_samples = 10000 # Number of simulations
mu, sigma = 10.0, 5.0 # Mean and standard deviation of blood pressure reduction (in mmHg)

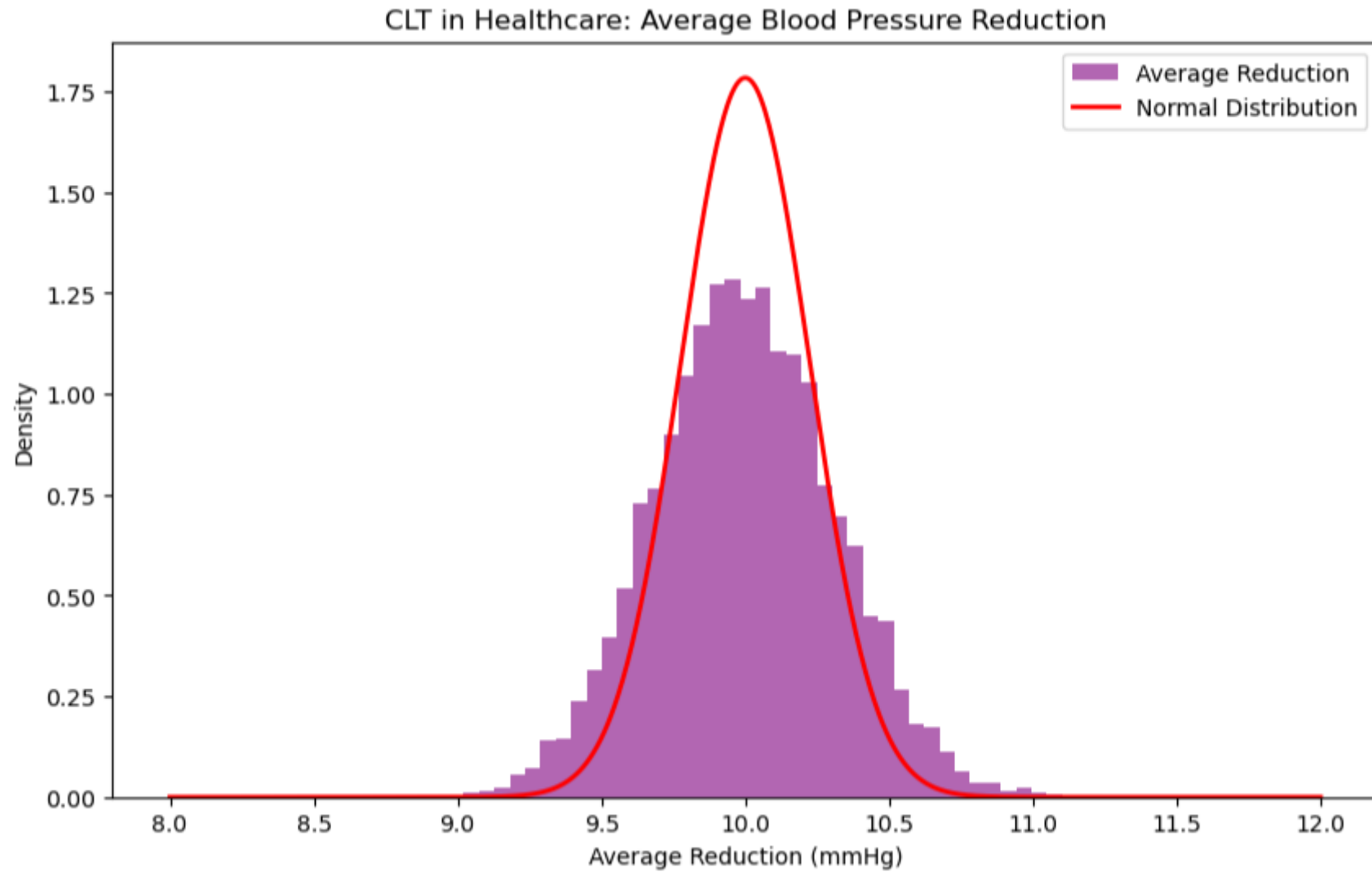
# Simulate blood pressure reductions
np.random.seed(42)
reductions = np.random.gamma(shape=2, scale=mu/2, size=(n_samples, n_patients)) # Gamma distribution
average_reductions = np.mean(reductions, axis=1) # Average reduction across patients

# Plot histogram of average reductions
plt.figure(figsize=(10, 6))
plt.hist(average_reductions, bins=50, density=True, alpha=0.6, color='purple', label='Average
Reduction')

# Overlay the theoretical normal distribution
x = np.linspace(mu - 2, mu + 2, 1000)
plt.plot(x, stats.norm.pdf(x, mu, sigma / np.sqrt(n_patients)), 'r--', lw=2, label='Normal
Distribution')

plt.title('CLT in Healthcare: Average Blood Pressure Reduction')
plt.xlabel('Average Reduction (mmHg)')
plt.ylabel('Density')
plt.legend()
plt.show()

```



- **CLT in Statistical Inference:**

CLT allows us to make inferences about a population based on a sample. It provides a way to estimate the population parameters, such as the *mean* and *Variance*, with a certain degree of confidence.

Example Calculation: Confidence Interval in Healthcare

Suppose in a drug trial, $n = 100$ patients with an average reduction in blood pressure of $\bar{x} = 10$ mmHg, with a sample variance $S^2 = 25$. Using the CLT, we can construct a 95% confidence interval for the true mean reduction μ as :

$$\bar{x} - z_{\alpha/2} \frac{\sigma}{\sqrt{n}} \leq \mu \leq \bar{x} + z_{\alpha/2} \frac{\sigma}{\sqrt{n}}$$
$$10 - 1.96 (0.5) < \mu < 10 + 1.96(0.5)$$
$$9.02 < \mu < 10.98$$

Where $Z_{0.025} = 1.96$

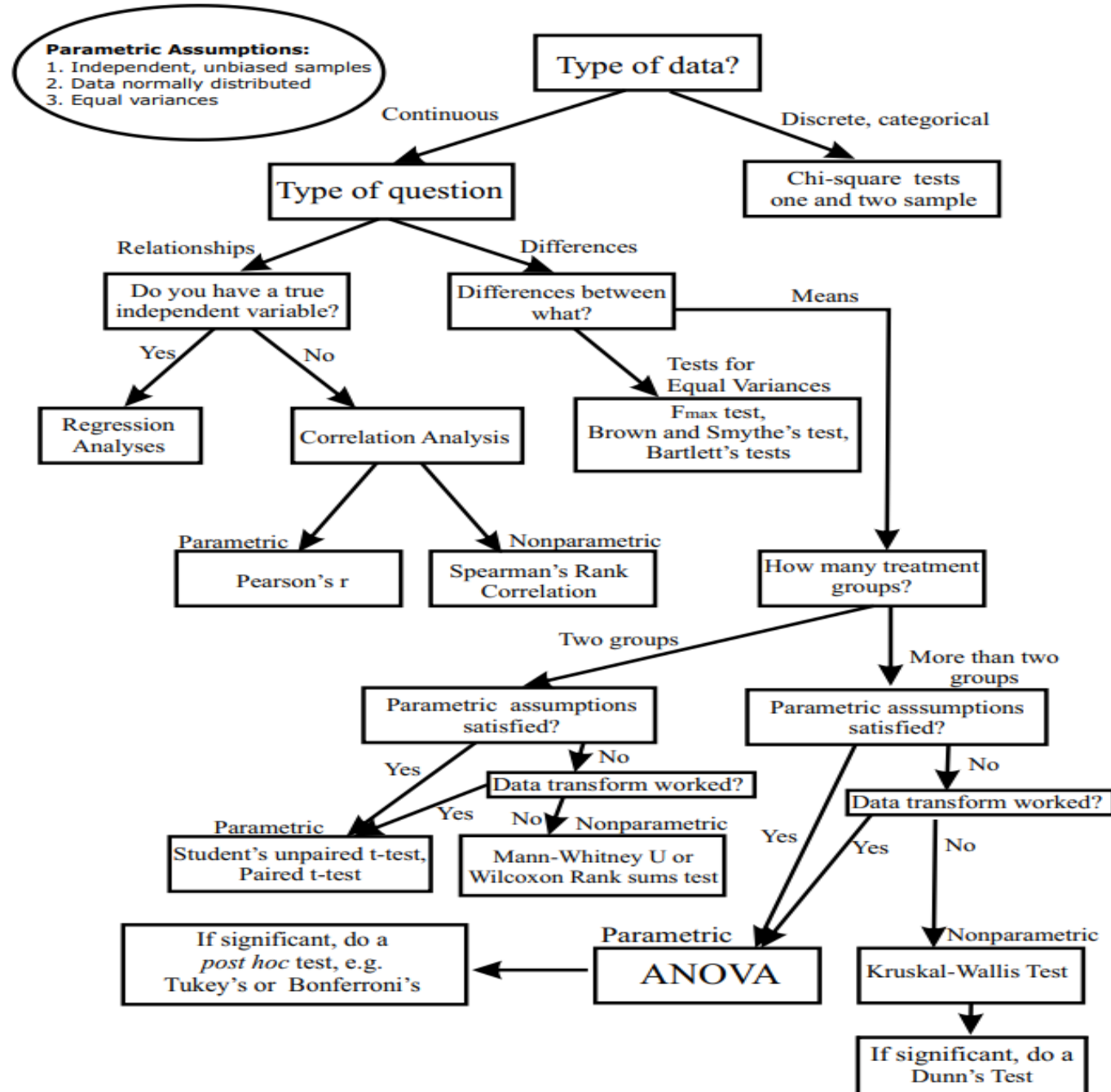
Description of Basic Functions used in R

<code>builtins()</code>	# List all built-in functions
<code>help()</code> or <code>?</code> or <code>??</code>	#i.e. <code>help(boxplot)</code>
<code>getwd()</code> and <code>setwd()</code>	# working with a file directory
<code>q()</code>	#To close R
<code>ls()</code>	#Lists all user defined objects.
<code>rm()</code>	#Removes objects from an environment.
<code>demo()</code>	#Lists the demonstrations in the packages that are loaded.
<code>demo(package = .packages(all.available = TRUE))</code>	#Lists the demonstrations in all installed packages.
<code>?NA</code>	# Help page on handling of missing data values
<code>abs(x)</code>	# The absolute value of "x"
<code>append()</code>	# Add elements to a vector
<code>cat(x)</code>	# Prints the arguments
<code>cbind()</code>	# Combine vectors by row/column (cf. "paste" in Unix)
<code>grep()</code>	# Pattern matching
<code>identical()</code>	# Test if 2 objects are *exactly* equal
<code>length(x)</code>	# Return no. of elements in vector x
<code>ls()</code>	# List objects in current environment
<code>mat.or.vec()</code>	# Create a matrix or vector
<code>paste(x)</code>	# Concatenate vectors after converting to character
<code>range(x)</code>	# Returns the minimum and maximum of x
<code>rep(1,5)</code>	# Repeat the number 1 five times

Description of Basic Functions used in R

rev(x)	# List the elements of "x" in reverse order
seq(1,10,0.4)	# Generate a sequence (1 -> 10, spaced by 0.4)
sequence()	# Create a vector of sequences
sign(x)	# Returns the signs of the elements of x
sort(x)	# Sort the vector x
order(x)	# list sorted element numbers of x
tolower(),toupper()	# Convert string to lower/upper case letters
unique(x)	# Remove duplicate entries from vector
vector()	# Produces a vector of given length and mode
formatC(x)	# Format x using 'C' style formatting specifications
floor(x), ceiling(x), round(x), signif(x), trunc(x)	# rounding functions
Sys.time()	# Return system time
Sys.Date()	# Return system date
getwd()	# Return working directory
setwd()	# Set working directory

Flow Chart for Selecting Commonly Used Statistical Tests



REFERENCES

- Practical Statistics for Data Scientists, 2020 Peter Bruce, Andrew Bruce, and Peter Gedeck
- Human-Centered Data Science, 2022 Cecilia Aragon, et al
- <https://www.geeksforgeeks.org/statistics-for-data-science/>

